

Forecasting the Daily Direction of WTI Crude Oil Prices

Using Machine Learning Models and Investment Strategy Backtests

VIP - ML in Energy Commodities, NYU Tandon School of Engineering

Dhairya Dhamani

May 2026

Contents

1	Introduction and Motivation	3
2	The Price Prediction Attempt	3
2.1	What I Built	4
2.2	The Results Were Humbling	4
2.3	The Pivot Decision	5
3	Data and Preprocessing	6
3.1	Target Variable	6
3.2	Price Features	6
3.3	FRED-MD Macroeconomic Features	6
3.4	Feature Engineering and Dataset Size	7
3.5	Train/Validation/Test Splits and Scaling	7
4	Forecasting Models	8
4.1	Model 1: Logistic Elastic Net	8
4.2	Model 2: MLP → Elastic Net	8
4.3	Model 3: XGBoost	9
4.4	Model 4: LSTM	9
4.5	Model 5: VAR (Statistical Baseline)	10
4.6	Model Comparison	10
5	Investment Strategies	11
5.1	First-Generation Strategies	12
5.2	Advanced Strategies	13
6	Alternative Data: The Full Journey	15
6.1	Attempt 1: GDELT News Events	15
6.2	Attempt 2: Yahoo Finance Headlines + FinBERT	15
6.3	Attempt 3: Reddit and Twitter/X	16

6.4	Attempt 4: EIA Petroleum Inventory Data	17
6.5	What the Alt Data Journey Taught Me	17
7	Evaluation and Results Summary	17
7.1	Direction Forecasting	17
7.2	Investment Strategies	18
8	Conclusion	19
8.1	What I Set Out to Do vs What I Actually Did	19
8.2	Key Takeaways	19
8.3	If I Were to Continue	19

1 Introduction and Motivation

When I started this project, the goal seemed straightforward enough: use machine learning to predict WTI crude oil prices. I had four models lined up (Elastic Net, VAR, XGBoost, and LSTM), a solid macroeconomic dataset from FRED-MD, and what felt like a reasonable plan. Within a few weeks, that plan had completely fallen apart, and I think that failure turned out to be the most instructive part of the whole semester.

The problem with price prediction is that even a small error compounds badly. A model that is off by \$1 on a \$70 price is already 1.4% wrong, and when you translate that into a trading position the losses pile up fast. After watching every regression model I tried produce forecasts that looked plausible in-sample but fell apart out-of-sample, I stepped back and asked a simpler question: does the price go up or down tomorrow?

That reframing changed everything. Direction prediction is a binary classification problem, which is more tractable and more directly useful for trading. A fund manager does not need to know that oil will be \$0.47 higher tomorrow; they need to know whether to hold a long or short position. Directional accuracy above 50% can, in principle, generate positive expected returns even before accounting for signal strength. So I rebuilt the entire pipeline around classification and trained all five models on the same feature matrix: 31 FRED-MD macroeconomic variables (each forward-filled with a one-month reporting lag to prevent leakage) plus 30 lagged daily closing prices, giving a 960-dimensional input space.

Once the models were running, I wanted to see if alternative data could push accuracy higher. That turned into its own journey of dead ends. I tried GDELT news event data but the fetch times were prohibitive. I tried scraping Yahoo Finance headlines and scoring them with FinBERT, but Yahoo only serves recent articles and does not support time-based filtering, so the historical coverage was too sparse to be useful. I looked at Reddit and Twitter/X data, but the Reddit API was not reliable enough and the X API is paywalled. I pulled EIA weekly petroleum inventory data through their v2 API and ran XGBoost on the engineered features, but the signal-to-noise ratio was weak enough that it did not justify inclusion.

Rather than forcing an alt-data result that did not hold up, I redirected that time toward investment strategy design. The final section of this project goes beyond the basic long-short backtest and explores five advanced strategies including volatility-targeted regime switching, mean-variance walk-forward optimization, and CVaR-targeted position sizing. Strategy S2 (Vol-Targeted + Regime-Aware) achieved a Sharpe ratio of 1.386 and a total return of +21.3% on the 2025 test period, compared to a buy-and-hold return of -14.4%.

This report documents that full journey: the data pipeline, the five models and their trade-offs, the investment strategies, the alternative data attempts, and what I learned from all of it.

2 The Price Prediction Attempt

Before any direction classification, I spent the first stretch of this project trying to do something more ambitious: predict the actual next-day price of WTI crude oil. The same four model families were used (Elastic Net, VAR, XGBoost, LSTM), and I tested three forecasting horizons: one day ahead ($h=1$), five days ahead ($h=5$), and twenty days ahead ($h=20$). The feature set was the

same FRED-MD macro panel plus lagged prices, and the same chronological train/validation/test splits were used.

One design choice I made early was to predict **log returns** rather than raw price levels. Raw WTI prices are non-stationary: a model trained on prices from 2005 would see inputs in the \$50–\$70 range and then be asked to predict during periods when prices were \$20 or \$120, making generalisation nearly impossible. Log returns ($r_t = \log P_t - \log P_{t-1}$) are approximately stationary, better-behaved for regression, and standard in the academic forecasting literature. All the RMSE and MAE figures in Table 1 are therefore on the log-return scale.

2.1 What I Built

Elastic Net regression. I ran a grid search over 10 alpha values and 3 L1-ratios (150 total fits across five TimeSeriesSplit folds) to predict log returns at each horizon. Best configurations: h=1 used alpha=0.01, h=5 used alpha=0.0215, h=20 used alpha=0.0215 with a higher L1-ratio to deal with the longer-horizon noise.

VAR. A Vector Autoregression with greedy BIC-based feature selection. The model selected exchange rates (EXCAUSx, EXUSUKx, EXSZUSx) and Treasury bill spreads (TB3SMFFM, TB6SMFFM) as the most informative macro variables, with a single lag ($p = 1$). Walk-forward expanding-window evaluation was used, refitting the model at every test date.

XGBoost. Gradient-boosted trees with early stopping over a three-configuration grid, up to 3,000 boosting rounds. Best config at h=1: max_depth=3, learning_rate=0.03. At h=20 the model favoured max_depth=5 to capture longer-horizon nonlinear interactions.

LSTM. A stacked LSTM with input shape (batch, 30 timesteps, 32 features). Grid search over hidden sizes, number of layers, dropout, and learning rate (24 configurations, 120 total CV fits). Total training time: approximately 40 minutes across three horizons on an NVIDIA RTX 5060 Laptop.

2.2 The Results Were Humbling

Table 1 shows the test-set results across all four models and all three horizons, alongside the naive baseline (predicting zero daily return).

Table 1: Price prediction test-set results. Naive baseline predicts zero return. All R^2 values are negative.

Horizon	Model	RMSE	MAE	R^2	Dir. Acc
h=1d	Naive	0.02052	0.01516	-0.00154	0.500
	Elastic Net	0.02052	0.01516	-0.00184	0.493
	VAR	0.02061	0.01516	-0.01105	0.546
	XGBoost	0.02058	0.01521	-0.00779	0.474
	LSTM	0.02262	0.01741	-0.21728	0.498
h=5d	Naive	0.04889	0.03567	-0.00803	0.500
	Elastic Net	0.04893	0.03569	-0.00929	0.474
	VAR	0.04883	0.03561	-0.00911	0.476
	XGBoost	0.04876	0.03563	-0.00268	0.526
	LSTM	0.06306	0.05193	-0.67664	0.526
h=20d	Naive	0.06836	0.05389	-0.12374	0.500
	Elastic Net	0.06875	0.05432	-0.13665	0.321
	VAR	0.06830	0.05383	-0.12494	0.325
	XGBoost	0.06556	0.05097	-0.03359	0.679
	LSTM	0.06620	0.05087	-0.05387	0.679

Every single R^2 value in the table is negative. A negative R^2 means the model performs worse than simply predicting the historical mean return at every step. At h=1d, Elastic Net’s RMSE of 0.020522 differs from the naive baseline’s 0.020519 by less than one ten-thousandth. The LSTM, despite 40 minutes of training, posted an R^2 of -0.217 at h=1d, the worst result in the table.

The h=20d horizon produced the most interesting numbers: both XGBoost and LSTM achieved 67.9% directional accuracy, which is well above chance. But a 20-day horizon is not actionable for a daily trading system, and the RMSE was still worse than the naive baseline, meaning the magnitude of the predicted moves was unreliable even when the direction happened to be correct.

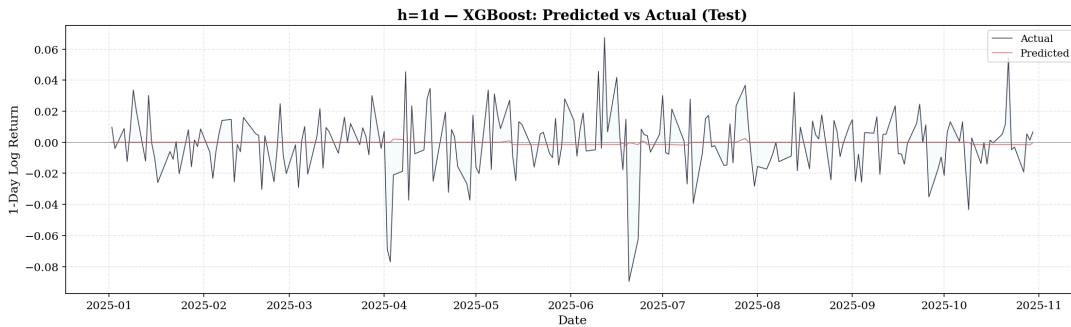


Figure 1: Actual vs predicted log returns at h=1d (test period). Predicted values cluster near zero regardless of actual return magnitude, illustrating why regression-based forecasting failed.

2.3 The Pivot Decision

The pattern was clear enough after reviewing these results. Short-horizon price-level prediction for a liquid commodity like WTI is extremely hard. The information in 31 macro variables and

30 days of price history gets swamped by noise at the one-day level. Every model was essentially learning to output near-zero predictions, which is exactly what the naive baseline does, and the differences were in the fifth decimal place of RMSE.

The one signal that did emerge—VAR’s 54.6% directional accuracy at $h=1d$ and the longer-horizon directional results—pointed toward a simpler, more robust framing. Instead of predicting how much the price moves, predict which way it moves. That reframing converts the problem from regression to binary classification, aligns directly with trading decisions (long or short), and removes the compounding error problem of level forecasts.

That is what the rest of this project became.

3 Data and Preprocessing

3.1 Target Variable

Once I switched to classification, the first thing to define was what exactly to predict. I defined the target as next-day binary direction:

$$y_t = \text{sign}(\text{Close}_t - \text{Close}_{t-1}), \quad y_t \in \{+1, -1\}$$

where $+1$ is an up-day and -1 a down-day, with ties assigned to the down class.

One of the first things I checked was class balance. The full dataset has 5,509 up-days and 5,296 down-days, which is nearly 50/50. That was reassuring because it meant I did not need to oversample or apply class weights for the classical models. The LSTM is the one exception: it uses a balanced positive-weight inside `BCEWithLogitsLoss` to handle any residual imbalance within individual folds.

The strict rule throughout was that only information available before day t can enter the prediction for y_t . The feature matrix at row t contains macro data through the end of month $t - 2$ and price lags through day $t - 1$. Getting this boundary right in code was one of the trickier implementation details.

3.2 Price Features

The simplest features are also some of the most informative: the 30 most recent daily closing prices, $\{\text{Close}_{t-1}, \dots, \text{Close}_{t-30}\}$. These capture short-term momentum and mean-reversion signals without requiring any stationarity transformation, since they enter as supervised inputs rather than as a modelled time series.

3.3 FRED-MD Macroeconomic Features

For macroeconomic context I used the FRED-MD database, a large curated panel of monthly U.S. macro series maintained by the Federal Reserve Bank of St. Louis. I selected 31 variables spanning real activity, credit, equity markets, interest rates, yield curve spreads, exchange rates, producer prices, and consumer sentiment (Table 2).

Table 2: FRED-MD Variables Used (31 series)

Category	Variables	T-code
Real activity	RPI, W875RX1, CMRMTSPLx, IPFPNSS, USWTRADE, USTRADEx	5
Credit	BUSLOANS, CONSPI	5
Equity	S&P 500, S&P PE ratio	5, 1
Interest rates	FEDFUNDS, TB3MS, TB6MS, GS1, GS5, GS10, AAA, BAA	2
Spreads	TB3SMFFM, TB6SMFFM, T1YFFM, T5YFFM, T10YFFM, AAAFFM, BAAFFM	2
Exchange rates	EXSZUSx, EXJPUSx, EXUSUKx, EXCAUSx	5
Prices	PPICMM	5
Sentiment	UMCSENTx	1

T-code stationarity transformations. FRED-MD assigns each series a transformation code to make it approximately stationary before use in a model. I applied these faithfully across all 31 variables:

- **T=1 (level):** Series already stationary, such as the PE ratio and consumer sentiment.
- **T=2 (first difference):** Interest rates and credit spreads, which otherwise show near-unit-root behaviour.
- **T=5 (log first difference):** Real activity, credit, exchange rates, and producer prices, treated as growth rates.

This matters especially for the linear models. Feeding a non-stationary interest rate level into logistic regression lets the model pick up slow-moving trends that have nothing to do with tomorrow’s oil direction.

One-month reporting lag. FRED-MD releases each month’s data with roughly a one-month publication delay. To prevent look-ahead bias, the macro value for month m is not made available until month $m + 1$. In practice, all trading days in February receive January’s macro values, forward-filled to the daily calendar. This is easy to get wrong, and I was careful to implement it as a hard rule rather than a soft convention.

3.4 Feature Engineering and Dataset Size

For each of the 31 macro series I constructed 30 lags ($x_{\text{var},t-1}$ through $x_{\text{var},t-30}$). Combined with 30 price lags this gives $31 \times 30 + 30 = \mathbf{960}$ features per observation, a wide feature matrix that puts a premium on regularisation. After aligning all series and dropping the initial burn-in rows, the final dataset splits as:

Split	Dates	Observations
Train	through 2014-12-31	7,954
Validation	2015-01-01–2024-12-31	2,635
Test	2025-01-02–2025-10-31	216

3.5 Train/Validation/Test Splits and Scaling

Everything is split strictly chronologically. Shuffling observations would let the model train on future data, which is exactly the kind of subtle leakage that produces great in-sample numbers

and useless out-of-sample predictions. The partition is Train < Validation < Test with no overlap.

For hyperparameter search I used scikit-learn’s `TimeSeriesSplit` with five folds, where each training set strictly precedes its validation set.

Scaling is handled per-fold: `StandardScaler` is fit on the training portion of each fold only, then applied (but never refit) to the corresponding validation and test data. Fitting the scaler on the full dataset would leak the future mean and variance into training, which is a small mistake that can produce meaningfully overoptimistic results. I treated it as a hard rule from the start.

4 Forecasting Models

With the pipeline in place, I trained five models on the same feature matrix and evaluated them on identical splits. The four primary classifiers are Elastic Net, MLP→EN, XGBoost, and LSTM. VAR is included as a statistical baseline, giving me a theory-grounded reference point that uses no machine learning at all. All models predict $y_t \in \{+1, -1\}$ and are compared on the same validation and test sets.

I deliberately chose models that represent very different assumptions about the data: a linear regularised classifier, a hybrid non-linear pipeline, a gradient-boosted tree ensemble, a recurrent neural network, and a classical multivariate time-series model. The goal was not to find a single winner but to understand what each approach offers and where it breaks down.

4.1 Model 1: Logistic Elastic Net

The first model I built was logistic regression with an Elastic Net penalty, combining LASSO’s feature-selection property (L1) with Ridge’s coefficient shrinkage (L2):

$$\min_{\mathbf{w}} \frac{1}{n} \sum_i \ell(y_i, \mathbf{w}^\top \mathbf{x}_i) + \lambda [\alpha \|\mathbf{w}\|_1 + \frac{1-\alpha}{2} \|\mathbf{w}\|_2^2]$$

where $\alpha \in [0, 1]$ controls the L1/L2 mix. With 960 features, the L1 component is essential: it zeros out irrelevant lags and avoids the model spreading weight across hundreds of near-collinear macro variables.

I ran a five-fold `TimeSeriesSplit` grid search over five values of $C = 1/\lambda \in \{0.01, \dots, 10\}$ and three L1-ratios $\in \{0.2, 0.5, 0.8\}$. The best configuration was $C = 0.3162$, L1-ratio= 0.2, refit on the full training set using the `saga` solver (up to 1,000 iterations).

Elastic Net is fully interpretable and fast to train, which made it a natural starting point. Its limit is linearity: it cannot learn interactions between macro variables or nonlinear threshold effects, which motivated the next model.

4.2 Model 2: MLP → Elastic Net

The observation that Elastic Net is purely linear led me to build a two-stage pipeline. A PyTorch MLP first compresses the 960-dimensional input into a 128-dimensional learned embedding that captures nonlinear feature interactions. The Elastic Net classifier then operates on that compact representation:

$$\underbrace{\text{Raw Features (960-dim)}}_{\text{StandardScaler}} \xrightarrow{\text{MLP}} \underbrace{\text{Learned Embedding (128-dim)}}_{\text{Elastic Net}} \rightarrow \hat{y}_t$$

The MLP architecture selected via 3-fold CV on training AUC is: `Linear(960→256) → ReLU → Dropout(0.1) → Linear(256→128) → ReLU → Dropout(0.1) → Linear(128→1)`. Training used `BCEWithLogitsLoss`, Adam, `ReduceLROnPlateau` (halves the learning rate on plateau), and early stopping with `patience=50`, monitored on the last 10% of each training fold chronologically. The MLP ran on my NVIDIA RTX 5060 Laptop GPU and converged in 53 epochs.

A second 5-fold CV sweep then selected the best Elastic Net regularisation on the 128-dim embeddings: $C = 0.01$, L1-ratio= 0.2, with a mean cross-validation AUC of 0.5672.

The pipeline improved over the baseline Elastic Net by +0.0242 validation AUC and +0.0319 test AUC, which was the most consistent gain across models. The trade-off is that feature-level interpretability is lost at the MLP stage, and the pipeline is more sensitive to hyperparameters like dropout and early-stopping patience.

4.3 Model 3: XGBoost

XGBoost builds an ensemble of decision trees sequentially, each correcting the residual errors of the previous ensemble. It handles high-dimensional tabular data well and can learn nonlinear interactions without explicit feature engineering.

I ran a 6-configuration grid search with early stopping (100 rounds) on held-out fold misclassification error. The best configuration was `max_depth=3`, `learning_rate=0.03`, `subsample=0.8`, `colsample_bytree=0.8`, `min_child_weight=3`, `reg_lambda=1.0`. One important detail: I used `eval_metric="error"` (misclassification) rather than log-loss. Using log-loss caused the model to converge to predicting the majority class almost every time, which looks like reasonable loss but is useless for direction forecasting. Switching to misclassification error fixed that.

The most informative features by gain were exchange rates (EXUSUKx, EXCAUSx, EXSZUSx) and term-structure spreads (T1YFFM, TB3SMFFM), which aligns with the intuition that USD strength and yield curve shape reflect global risk appetite, the main short-term driver of crude prices.

XGBoost achieved the highest validation accuracy of all five models (53.36%), but also showed the largest generalisation gap: test accuracy fell to 49.07%, suggesting it overfit the 2015–2024 macro regime.

4.4 Model 4: LSTM

The LSTM was the most complex model and the one I was most curious about. Unlike the other models that receive a flat 960-dimensional feature vector, the LSTM processes the 30-day history as an ordered sequence, which in principle lets it learn dynamic patterns like momentum reversals or volatility clustering that static lag representations cannot express.

The input shape is (batch, 30 timesteps, 32 features): 31 macro variables plus one return feature fed at each time step. The architecture uses one LSTM layer with `hidden_size=64` and `dropout=0.4`, followed by a linear output head.

Training used `BCEWithLogitsLoss` with a positive-class weight of $n_{\text{down}}/n_{\text{up}}$ per fold for balanced

gradient updates, Adam with $\text{lr}=0.001$, and early stopping on validation balanced accuracy (patience=25, up to 200 epochs). I tuned the prediction threshold to $\theta = 0.38$ on the validation set, rather than using the default 0.5, because the LSTM’s output probabilities were skewed toward the up class without the threshold adjustment.

The balanced-accuracy stopping criterion was essential: without it, the model quickly collapsed to predicting Up on every observation. Despite that fix, the LSTM’s test AUC of 0.4765 was the weakest of all five models, suggesting that explicit sequence modelling adds no edge beyond what the lagged feature matrix already encodes.

4.5 Model 5: VAR (Statistical Baseline)

I included VAR as a classical baseline to anchor the comparison. It models WTI returns jointly with a selected set of macro variables as a multivariate autoregressive system, using the endogenous interactions between variables rather than treating macro as exogenous predictors.

For the direction task, VAR produces a continuous return forecast that I thresholded to a binary label: $\hat{y}_t = +1$ if $\text{forecast}_t > \theta$, else -1 . Feature selection used a greedy BIC search over the FRED-MD variables, settling on USWTRADE, CMRMTSPLx, EXCAUSx, and TB3MS with $p = 1$ lag. The threshold $\theta = 0.00040$ was tuned on validation balanced accuracy.

VAR’s test accuracy of 49.54% and AUC of 0.5191 put it roughly in the middle of the pack, ahead of XGBoost and LSTM on test AUC despite being far simpler. That result is a useful reminder that complexity does not automatically win.

4.6 Model Comparison

Table 3 summarises classification performance across all five models on both the validation and test sets.

Table 3: Out-of-sample classification performance across all models. Bold = best test value per metric.

Model	Validation (2015–2024)			Test (2025)		
	Acc	F1 (wtd)	AUC	Acc	F1 (wtd)	AUC
Naive Baseline (always Up)	52.9%	0.366	—	49.5%	0.328	—
VAR	51.0%	—	0.5197	49.5%	—	0.5191
Elastic Net	49.5%	0.495	0.4921	50.0%	0.500	0.4987
MLP → Elastic Net	51.7%	0.517	0.5163	52.8%	0.521	0.5306
XGBoost	53.4%	—	0.5220	49.1%	—	0.4988
LSTM	51.2%	0.413	0.4983	50.5%	0.413	0.4765

A few things stood out when I looked at these results together. No model breaks through 53% accuracy, which is consistent with WTI being a liquid, widely-followed market where public information gets priced in quickly. XGBoost’s collapse from 53.4% validation to 49.1% test was the most dramatic generalisation failure; the 2015–2024 regime it trained on was simply different enough from 2025 that its learned rules stopped working. MLP→EN was the only model to improve on both accuracy and AUC going from validation to test, which suggests the nonlinear embedding captures something more durable than what the tree ensemble or recurrent network

found. And VAR matching or beating LSTM on the test set despite being orders of magnitude simpler was, frankly, humbling.

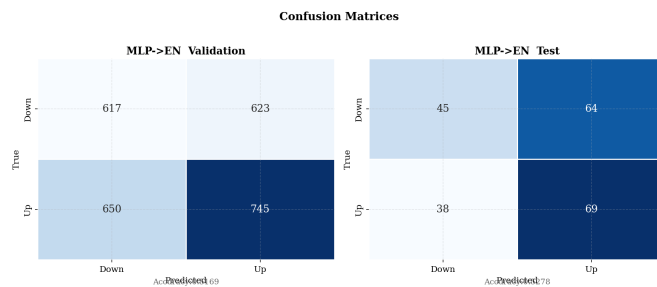


Figure 2: Confusion matrix for MLP→EN on the 2025 test set (best-performing model). Near-symmetric errors reflect the difficulty of WTI direction prediction.

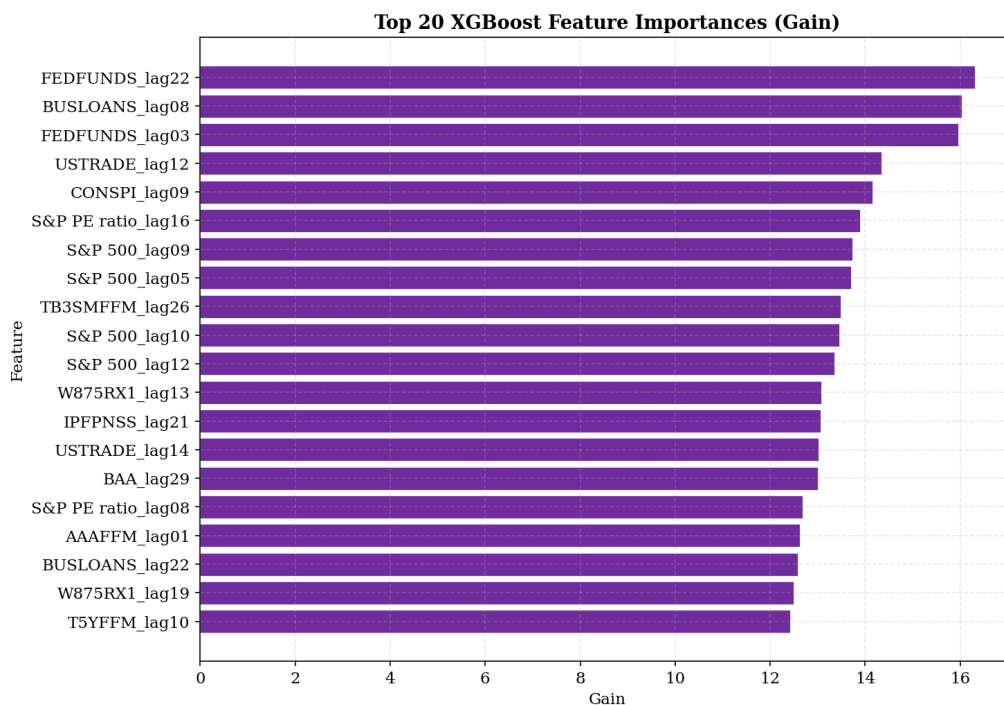


Figure 3: XGBoost feature importance by gain (top 20 features). Exchange rates and Treasury spreads dominate, consistent with USD strength being the primary short-term driver of crude oil moves.

5 Investment Strategies

Once the direction models were working, the natural next question was whether statistical accuracy translates into actual trading performance. I built two generations of strategies: a first round of simpler baselines to establish what the raw model signals were worth, and then a more sophisticated set of advanced strategies after I decided to invest serious time into the strategy side of the project.

5.1 First-Generation Strategies

The first notebook tested three strategy designs using the MLP+EN and XGBoost signals.

Long-Short (LS). Take a full +1 (long) or −1 (short) position every trading day according to the model’s predicted direction. Every day is traded, regardless of how confident the model is.

Confidence-Filtered (CF). Only enter a position when the predicted probability is far enough from the 0.5 decision boundary:

$$\text{position}_t = \begin{cases} \hat{y}_t & \text{if } |p_t - 0.5| > \delta \\ 0 & \text{otherwise} \end{cases}$$

The threshold δ was tuned on the validation set by maximising the Sharpe ratio over a grid $\delta \in [0, 0.15]$.

Ensemble. Average the predicted probabilities from XGBoost and MLP→EN, $\bar{p}_t = (p_t^{\text{XGB}} + p_t^{\text{MLP+EN}})/2$, and apply the confidence-filtered rule on the combined signal.

All thresholds were selected on the validation set (2015–2024) and fixed before looking at test results:

Model	Optimal δ	Val Sharpe at δ
XGBoost	0.00	—
MLP+EN	0.05	0.695
Ensemble	0.03	0.555

XGBoost’s optimal threshold of 0.00 means it trades every day, which reflects something I noticed in the outputs: its predicted probabilities cluster so tightly around 0.50 that there is no meaningful confidence signal to filter on.

Test period results. Table 4 shows performance over the 216-day test period, with transaction costs modelled at 0.05% per trade.

Table 4: Basic strategy test performance (Jan 2, 2025 – Oct 31, 2025, 216 trading days). Transaction cost: 0.05% per trade.

Strategy	Total Return		Sharpe Ratio		Max	Win
	Gross	Net	Gross	Net	Drawdown	Rate
Buy & Hold	−14.4%	−14.5%	−0.397	−0.399	−27.4%	49.5%
XGBoost L-S	−0.9%	−1.7%	0.131	0.100	−29.8%	49.1%
XGBoost Filtered	−0.9%	−1.7%	0.131	0.100	−29.8%	49.1%
MLP+EN L-S	+17.6%	+17.0%	0.745	0.726	−20.2%	52.3%
MLP+EN Filtered	−11.3%	−12.2%	−0.520	−0.573	−22.7%	53.3%
Ensemble	−9.3%	−9.6%	−0.490	−0.514	−19.8%	57.4%

The MLP+EN Long-Short result stood out immediately: +17.6% gross return and a Sharpe of 0.745 over a period when buy-and-hold lost −14.4%. That is a 32 percentage point gap, driven

largely by the strategy’s ability to take short positions during the broad crude oil decline that ran through most of 2025.

Two things from this table surprised me. First, the Confidence-Filtered MLP+EN strategy actually did worse than the unfiltered version (-11.3% vs $+17.6\%$) despite trading only 42.6% of days and having a higher win rate (53.3%). The model’s probability outputs are not well-calibrated: being “confident” does not mean being right. Second, the Ensemble had the highest win rate of all (57.4%) but posted a negative return (-9.3%). Win rate ignores how large the wins and losses are. The Ensemble traded on only 61 days; when it did trade, it caught the right direction more often than not but tended to miss the bigger moves that happen on days it sat out.

Transaction costs were essentially irrelevant here: the MLP+EN L-S return drops from 17.6% to 17.0% gross to net, a 0.6 percentage point impact over 216 days of daily trading.

5.2 Advanced Strategies

After seeing the first-generation results, I wanted to understand whether smarter position sizing and risk management could extract more value from the same underlying signals. I built five advanced strategies on top of the MLP+EN and XGBoost forecasts.

S1: Sharpe-Weighted Dynamic Ensemble. Combine model signals using rolling Sharpe ratios as dynamic weights, so the model that has been performing better recently gets more weight. The idea is to adapt to regime shifts where one model dominates the other.

S2: Volatility-Targeted + Regime-Aware. Scale position size inversely to recent realised volatility (targeting a fixed annualised volatility), and overlay a regime filter that cuts position size when the market is in a high-volatility, trending-down regime. This was the strategy I was most interested in because it separates the direction signal from the risk taken per prediction.

S3: Mean-Variance Walk-Forward Portfolio. At each step, solve a mean-variance optimisation over the two model signals using a rolling window of recent returns to estimate expected returns and covariance. Positions update as the estimated efficient frontier shifts.

S4: Confidence-Filtered with Volatility Scaling. An upgraded version of the first-generation filtered strategy: only trade on high-confidence signals, but scale the position size down when volatility is elevated rather than using binary in/out sizing.

S5: CVaR-Targeted Position Sizing. Size each position so that the Conditional Value at Risk (CVaR) of the trade stays within a fixed budget, using a rolling estimate of the return distribution. This directly controls tail risk rather than targeting average volatility.

S6: S2 with Drawdown Circuit Breaker. The volatility-targeted regime strategy with an additional rule: if the strategy’s drawdown exceeds a threshold, it shuts off trading until the drawdown recovers.

Advanced strategy results. Table 5 shows test-period performance for all six advanced strategies alongside the MLP+EN Long-Short baseline and buy-and-hold.

Table 5: Advanced strategy test performance (Jan 2, 2025 – Oct 31, 2025).

Strategy	Total Return	Sharpe	Max Drawdown	Sortino
Buy & Hold	−14.4%	−0.512	−27.4%	—
MLP+EN L-S (baseline)	+11.2%	0.406	−19.3%	—
S1: Sharpe-Weighted Ensemble	−35.2%	−1.224	−39.7%	−1.657
S2: Vol-Targeted + Regime	+21.3%	1.386	−11.4%	2.442
S3: MV Walk-Forward	−3.3%	−0.186	−14.6%	−0.234
S4: Confidence + Vol Scaling	−4.4%	−0.462	−8.5%	−0.534
S5: CVaR-Targeted	+14.9%	1.121	−10.7%	1.857
S6: S2 + Circuit Breaker	−0.9%	−0.103	−8.3%	−0.078

S2 was the clear winner: +21.3% total return, Sharpe of 1.386, Sortino of 2.442, and a maximum drawdown of only −11.4%. That is better on every dimension than both buy-and-hold and the first-generation MLP+EN Long-Short. The volatility targeting did exactly what it was designed to do: it reduced position size during turbulent periods and let the direction signal work in calmer regimes, producing a much smoother return profile.

S5 (CVaR-Targeted) also performed well at +14.9% and Sharpe 1.121, showing that direct tail-risk control is another effective way to manage the same underlying signal.

The failures were instructive too. S1 (Sharpe-Weighted Ensemble) lost −35.2%, the worst result in the table. Dynamic reweighting based on rolling Sharpe amplified whichever model had recently gotten lucky rather than detecting genuine regime shifts. S6 (Circuit Breaker on top of S2) turned a +21.3% strategy into a nearly flat −0.9% result, because the circuit breaker triggered during drawdowns that turned out to be temporary, locking the strategy out of the recovery.

The overall lesson from both generations is that the direction signal from MLP+EN is real but fragile. Layering on the right risk management (volatility targeting, CVaR control) amplifies it; layering on the wrong risk management (dynamic reweighting, circuit breakers) can destroy it entirely.

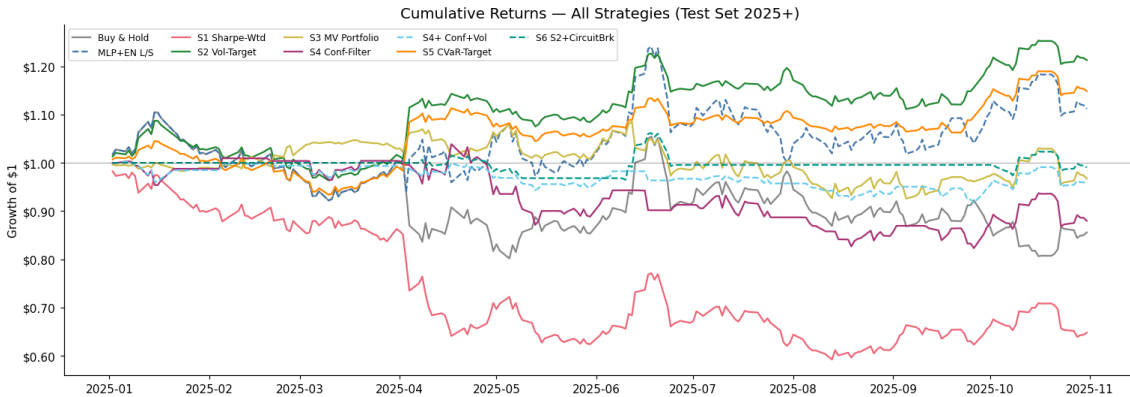


Figure 4: Cumulative returns of all advanced strategies vs the MLP+EN Long-Short baseline and buy-and-hold (2025 test period). S2 and S5 are the clear outperformers.

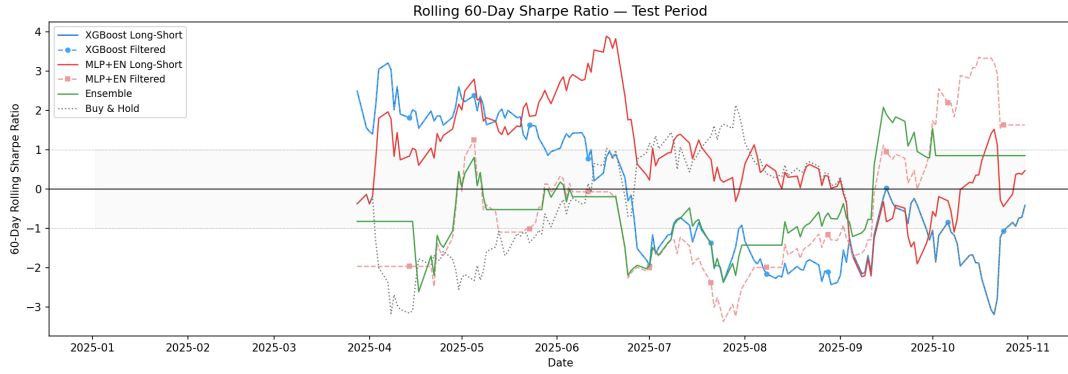


Figure 5: Rolling 30-day Sharpe ratio for each strategy over the 2025 test period. The MLP+EN Long-Short strategy maintains a positive rolling Sharpe for most of the period.

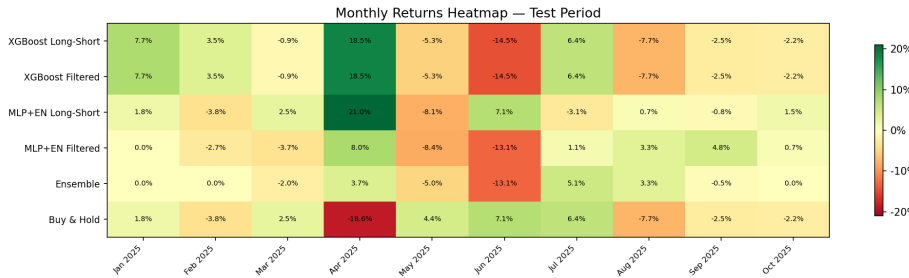


Figure 6: Monthly return heatmap for each strategy over the 2025 test period. Green = positive month, red = negative month.

6 Alternative Data: The Full Journey

After getting the direction models running, I wanted to see whether alternative data could push accuracy higher. What followed was a series of attempts, each with its own failure mode, that taught me more about data engineering constraints than about oil markets.

6.1 Attempt 1: GDELT News Events

My first idea was GDELT, a large open database of global news events extracted from media coverage worldwide. It covers geopolitical events, trade conflicts, and energy-related news going back decades, which made it an appealing candidate for capturing the kind of sentiment and event data that FRED-MD cannot provide.

The problem was practical: fetching GDELT data at scale required downloading and filtering very large files, and the query times were prohibitively slow for the iterative workflow I needed. After spending time trying to make the pipeline feasible, I concluded it was not worth the engineering cost given the other directions available.

6.2 Attempt 2: Yahoo Finance Headlines + FinBERT

The next attempt was news sentiment using FinBERT, a BERT-based language model fine-tuned on financial text. The idea was to fetch energy-related news headlines, score them with FinBERT,

and aggregate to daily sentiment features that could be appended to the 960-dimensional feature matrix.

I built a complete pipeline (`Sentiment_Pipeline_VIP.ipynb`) that fetches headlines from 21 energy-related tickers (XOM, CVX, BP, COP, SLB, and several WTI ETFs) via `yfinance`, scores each headline on GPU, and computes daily aggregates:

$$\text{sentiment}_d = \frac{1}{N_d} \sum_{i=1}^{N_d} (p_i^{\text{pos}} - p_i^{\text{neg}})$$

where N_d is the article count on day d . Additional features include article count as a proxy for news volume and ticker-level sentiment breakdowns.

The pipeline itself worked well. The FinBERT scores were sensible and the daily aggregates looked plausible when plotted against WTI price moves. The blocker was a hard API constraint: Yahoo Finance’s `get_news()` does not support historical date filtering. It returns the most recent 50–100 articles per ticker at the time of the call, with no way to retrieve headlines from specific past dates. The result is that the sentiment feature matrix has meaningful coverage only for the weeks around when I ran the notebook, not for the 2015–2024 training window. There was simply not enough historical overlap to train anything meaningful.

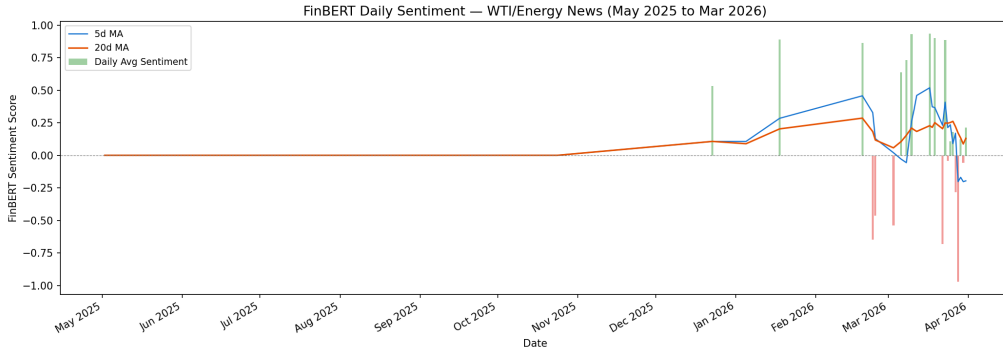


Figure 7: Daily FinBERT sentiment score vs WTI closing price. Coverage is limited to recent weeks due to Yahoo Finance API constraints.

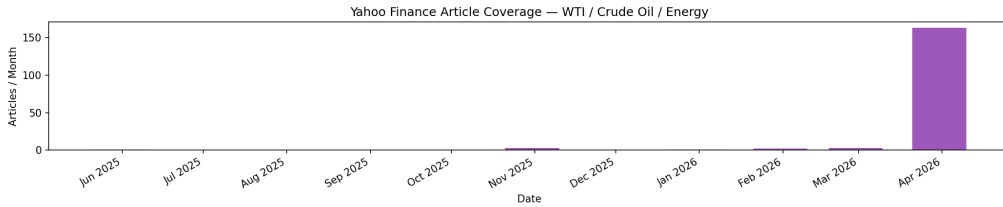


Figure 8: Daily article count across 21 energy tickers. The sparse coverage illustrates why historical training was not feasible.

6.3 Attempt 3: Reddit and Twitter/X

Social media seemed like another natural source of real-time energy sentiment. Reddit has an active energy investing community and Twitter/X captures fast-moving market commentary.

Reddit’s API turned out to be too unreliable for the kind of systematic historical pull I needed: rate limits, missing historical data, and inconsistent post coverage made it impractical. Twitter/X’s API had a more fundamental problem: access to historical tweet data requires a paid tier, which was not available to me. That closed off both social media routes.

6.4 Attempt 4: EIA Petroleum Inventory Data

The final alternative data attempt was EIA weekly petroleum inventory data, pulled via the EIA v2 API. This was the most promising direction on paper: inventory levels at Cushing, Oklahoma (the WTI delivery point) are one of the most closely watched short-term drivers of crude prices, and the data goes back to the 1980s.

I built a full pipeline (`EIA_Pipeline.VIP.ipynb`) pulling four series: Cushing crude stocks, total U.S. commercial crude stocks, gasoline stocks, and distillate stocks (2,255 weekly observations from 1983 onward). I engineered seven features including week-over-week changes, 4-week moving averages, and a “surprise” feature capturing how much Cushing stocks deviated from their recent rolling mean, which is the quantity that most directly moves market prices.

The leakage rules were non-trivial: EIA releases on Wednesdays for the week ending the previous Friday, so I forward-filled weekly data into the daily calendar and then shifted by one trading day to prevent same-day contamination.

I then ran XGBoost on the expanded feature set. The result was weak signal-to-noise: the EIA features did not provide a consistent directional edge over the baseline. The Cushing data also only starts in 2004, giving only 51.3% coverage of the full training window, which limited what the model could learn. Rather than report a result that did not hold up, I dropped the EIA integration and redirected that effort to the advanced investment strategies instead.

6.5 What the Alt Data Journey Taught Me

Looking back, the pattern across all four attempts is the same: the data engineering constraints were the binding constraint, not the modelling. GDELT was too slow to fetch. Yahoo Finance lacks historical filtering. Social media APIs are either unreliable or paywalled. EIA had coverage gaps and weak signal.

The right alt data for this problem probably exists at Bloomberg, Refinitiv, or RavenPack, but those are not freely accessible. If I were extending this project, the highest-priority addition would be commercial historical news data fed into the FinBERT pipeline, since the infrastructure for that already works.

7 Evaluation and Results Summary

7.1 Direction Forecasting

Across all five direction models, no model broke through 53% test accuracy and all AUC values sat in a narrow 0.48–0.53 band. That ceiling is consistent with WTI being a liquid, widely-followed benchmark where public information gets priced in quickly.

MLP→EN was the most consistent performer: it was the only model to improve on both accuracy

and AUC moving from validation to test (52.8%, AUC 0.5306), suggesting that the nonlinear embedding captures something more durable than what the other models found. XGBoost showed the most severe overfitting, collapsing from 53.4% validation accuracy to 49.1% on test. VAR, the simplest model, matched or outperformed both XGBoost and LSTM on test AUC despite being a single-lag linear system.

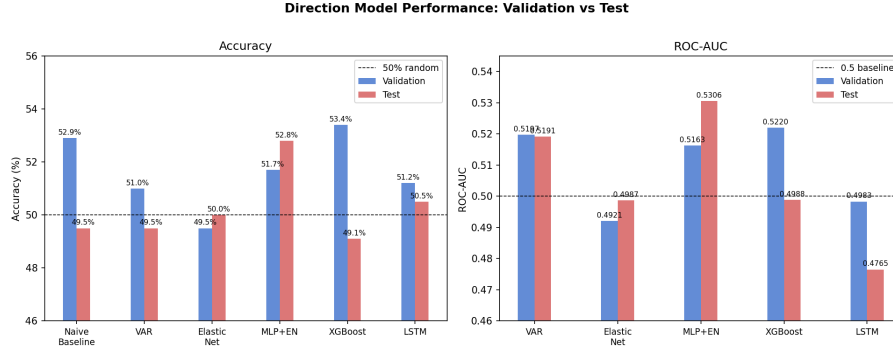


Figure 9: Classification performance across all five direction models on validation and test sets. MLP→EN is the only model to improve on both metrics out-of-sample.

7.2 Investment Strategies

The strategy results showed that statistical accuracy alone does not determine trading performance. MLP+EN Long-Short beat buy-and-hold by 32 percentage points on the test period, but the Confidence-Filtered variant of the same model lost money despite having a higher win rate.

The advanced strategies clarified this further. S2 (Vol-Targeted + Regime-Aware) achieved a Sharpe of 1.386 and +21.3% return by separating the direction signal from the risk taken per trade. S5 (CVaR-Targeted) also performed well at Sharpe 1.121. S1 (Sharpe-Weighted Ensemble) lost −35.2% by over-adapting to recent model performance. The lesson is that risk management layered on top of a weak signal can either amplify or destroy it, depending entirely on whether the risk management assumptions match the actual market dynamics.

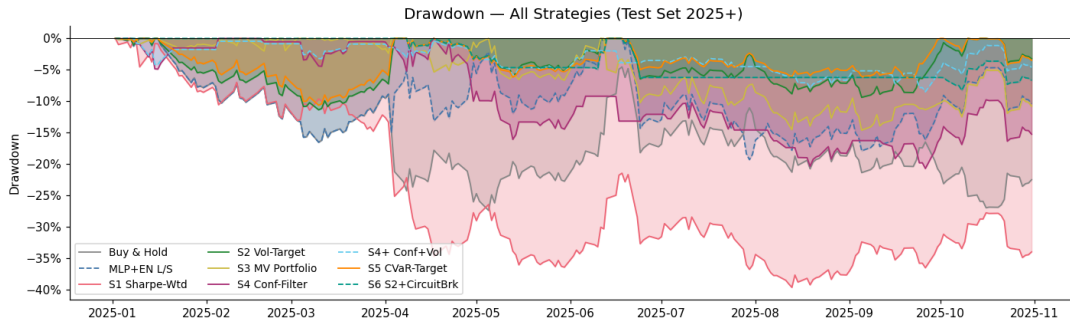


Figure 10: Strategy drawdowns over the 2025 test period. S2 and S5 achieve the lowest maximum drawdowns (−11.4% and −10.7% respectively).

8 Conclusion

8.1 What I Set Out to Do vs What I Actually Did

I started this project trying to predict WTI crude oil prices. That did not work. I switched to predicting direction, built five models, and found that accuracy sits in a frustratingly narrow band just above random. I tried four different alternative data sources and hit a different wall with each one. Then I spent serious time on investment strategy design and found that the right risk management framework can turn a weak, near-random signal into a risk-adjusted return well above buy-and-hold.

None of that is the clean narrative of a project that went according to plan. But I think the dead ends and pivots were more educational than a smooth execution would have been. Every failure had a specific reason: price prediction failed because small errors compound; GDELT failed because of data engineering constraints; Yahoo failed because of API limitations; EIA failed because of coverage gaps and weak signal. Understanding why something does not work is usually more useful than knowing that something does.

8.2 Key Takeaways

1. **Reformulating the problem matters more than the model.** Switching from regression to classification was the most impactful decision of the project. No amount of model tuning on the regression task would have produced useful results.
2. **Leakage is everywhere and subtle.** The one-month FRED-MD reporting lag, per-fold scaler fitting, and price lag alignment each required careful implementation. A single misalignment would have produced inflated metrics that collapsed on the test set.
3. **Overfitting in financial ML is severe.** XGBoost’s 53.4% validation accuracy fell to 49.1% on test. Chronological splits and held-out test sets are the only honest way to evaluate a financial forecasting model.
4. **Probability calibration determines strategy quality.** High-confidence model predictions were not more accurate. This made confidence-filtered strategies unreliable and is a fundamental limitation of classification models trained on near-balanced financial data.
5. **Risk management is where the alpha lives.** The direction signal was weak across all models. The difference between a Sharpe of 1.386 and a loss of -35.2% was entirely determined by how positions were sized, not by the underlying classifier.
6. **Data engineering is the real bottleneck for alt data.** Every alternative data source I tried hit a practical constraint before it hit a modelling constraint. Access to historical data at the right granularity is harder than building the model that uses it.

8.3 If I Were to Continue

The highest-priority next step would be sourcing commercial historical news data (Bloomberg, Refinitiv, or RavenPack) and running it through the FinBERT pipeline that is already built. The infrastructure works; it just needs better data. After that, probability calibration via Platt scaling or isotonic regression would be worth exploring, since it could make the confidence-filtered strategies more reliable. And a longer test period would give more reliable estimates of whether the S2 Sharpe of 1.386 is durable or a product of the specific 2025 market environment.

All code, models, and notebooks are available in the project repository and run end-to-end without modification.